Exam Project

Maciej Jałocha (macja@itu.dk)

08.05.2026

1 Introduction

With a population of more than 600,000 residents, Copenhagen has 380 km of bicycle lanes, and 40% of commuters travel by bicycle [1]. In such a city, the ability to predict bicycle traffic could help, for example, with real-time traffic management and with estimating CO_2 savings from cycling.

The purpose of this project is to investigate whether a regularised linear model based on weather, calendar, and lag features is suitable for predicting near-future bicycle traffic in Copenhagen.

To do so, I analyse ridge regression for one-week-ahead predictions, both quantitatively and qualitatively. Guided by the baseline results from linear regression, I introduce engineered features. I also examine the potential of lag features. These features, together with ridge regression and a cross-validated λ, form the best model found. I then assess how generalisable this model is. Finally, I discuss whether the model's performance is practically suitable.

2 Data

The data consist of three components: bike counts, Danish public holidays, and Copenhagen weather data. The date range is 01-01-2021 to 31-12-2023 for all three sources.

The bike-count dataset contains hourly bicycle counts recorded at six locations in the city: Amagerbrogade, Dronning Louises Bro, Torvegade, Noerrebrogade, H.C. Andersens Boulevard, and Langebro [2]. Each station has 26,280 total hourly observations, of which approximately 2% are missing due to sensor outages; station-specific counts are reported in Appendix Table 2.

The Copenhagen weather dataset contains hourly records of temperature (°C), wind speed (m/s), and precipitation (mm) [3].

3 Methods

3.1 Splits and Evaluation

The data are split into training and test sets. The first two years are used for training, and the third year is used for testing. A time-ordered split prevents leakage between the two sets when lag features are used. Using full-year splits also makes the data more balanced with respect to seasonal cycles, as the model sees two instances of the same calendar period during training and is evaluated on the third.

Time series cross-validation is used [5, 6]. In each split, the model is trained on all previous folds and evaluated on the next chronological fold. This allows more data to be used for training, as a separate validation set is unnecessary. The training and test periods contain approximately 104 and 52 calendar weeks, respectively; the test evaluation uses 7-day rolling folds, while the cross-validation folds are approximately weekly because `TimeSeriesSplit` divides the available rows evenly.

The evaluation folds are approximately one week long, for two reasons. First, the available weather data are observations, whereas a deployed model would rely on forecasts, which limits real-world applicability. In particular, an hourly forecast for Copenhagen may differ substantially from the observed weather. This mismatch between forecast and observed data is reduced by limiting the prediction window to 7 days. Secondly, weekly retraining is likely feasible, since the data are collected digitally, and linear models are inexpensive to train.

When evaluating on the test set, the model is initially trained on two years of data and then retrained each subsequent week. This allows the model to incorporate more recent data and to use lag features. More details are in Appendix Table 3.

3.2 Handling of Missing Data

Missing bike counts were dropped for models without lag features. Missing 2% was not deemed a concern, especially since there were only 106 cases of consecutive missing observations. For the temporal model, linear interpolation was used to ensure that each day had a non-empty lag feature. This is a pragmatic choice for the present analysis; a stricter deployment simulation should construct missing lag values using only information available before the prediction time.

3.3 Linear and Ridge Regressions

Let y_i denote the bicycle count at hour i, and let $x_i \in \mathbb{R}^p$ denote the corresponding vector of weather, calendar and lag features. The objective is to predict y_i from x_i using a linear model of the form

$$\hat{y}_i = \beta_0 + x_i^T \beta,$$

where β_0 is the intercept and $\beta \in \mathbb{R}^p$ is the coefficient vector.

In ordinary linear regression (OLS), the coefficients are estimated by minimising the residual sum of squares (RSS):

$$\min_{\beta_0,\beta} \sum_{i=1}^{n} \left(y_i - \beta_0 - x_i^T \beta\right)^2$$

This model is appropriate when the conditional mean of the response is approximately linear in the features. In addition, classical inference assumes that the errors are zero-centred, approximately normally distributed, and exhibit limited dependence [4, Ch. 3, pp. 69–109]. Although these assumptions are not fully satisfied here, linear models often offer competitive predictive performance [4, Ch. 3, pp. 69–109][4, Ch. 6, p. 229], and are easy to configure and interpret. Linear regression on raw features is used to guide the feature engineering. Ridge regression is then tried to reduce unstable coefficient estimates caused by multicollinearity in OLS [4, Sec. 3.3.3, pp. 106–109].

Ridge regression extends linear regression by adding an L_2 penalty to the objective,

$$\min_{\beta_0,\beta} RSS + \lambda \sum_{j=1}^{p} \beta_j^2,$$

where $\lambda \geq 0$ is a tuning parameter. The penalty shrinks the coefficients towards zero and reduces their variance, which is useful in the presence of multicollinearity [4, Sec. 6.2.1, pp. 240–244]. This is relevant here as some of the engineered features are correlated by construction. Thus ridge is expected to have more stable coefficients and better out-of-sample predictions. Also, ridge regression is most useful when many features contribute to the response [4, Sec. 6.2.1, pp. 240–244], which is plausible here because temperature, wind, rain and time may all explain bicycle traffic.

Before using ridge regression, standard scaling is applied to the features. For each feature x_j, the scaled version is computed as

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j},$$

where μ_j and σ_j denote the mean and standard deviation of feature j. These are computed on the training data. Scaling is necessary in the ridge regression as it prevents the penalization on a basis of a feature scale.

3.3.1 Best Tuning Parameter Search

In order to find the best λ parameter for ridge regression, I iterate over a logarithmic grid of 100 values between 10^{-2} and 10^{8}, scaled by the standard deviation of the response variable. The tuning parameter is selected by cross-validation, which is the standard approach for ridge regression [4, Sec. 6.2.3, pp. 252–253; Sec. 6.5.2, p. 273].

3.4 Feature Engineering

Additional features were engineered from the timestamp, holiday information, and past bicycle counts. Their effect on predictive performance is evaluated in the Experiments and Results section.

From each timestamp, cyclic representations of hour, day of week, and month were constructed. For a periodic variable t with period P, the cyclic encoding is given by $\sin\left(\frac{2\pi t}{P}\right)$ and $\cos\left(\frac{2\pi t}{P}\right)$. Accordingly, hour, day of week, and month were encoded with periods 24, 7, and 12, respectively.

Two binary calendar indicators were also included: one indicating whether an observation falls on a weekend and one indicating whether the date is a public holiday.

In the temporal model, an additional lag feature was included, equal to the bicycle count observed 7 days (168 hours) earlier.

Finally, these features were expanded with squared terms and pairwise interaction terms between them, totalling of 77 features for the improved model 4.2, and 90 features for the temporal model 4.2.1. These are referred as "engineered features".

Details on the pipeline implementation can be found in Appendix A.3.

3.5 Evaluation Metric

Model performance is evaluated using the root mean squared error, $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$. The $RMSE$ measures the typical prediction error in the same units as the response variable, here hourly bicycle counts, which makes it directly interpretable. Compared with the mean absolute error (MAE), the $RMSE$ penalises large errors more strongly because of the squaring of the residuals and therefore favours models with fewer large mismatches.

When evaluating a model, I report the mean RMSE across all validation folds ("mean RMSE").

4 Experiments and Results

First, I train a linear regression on raw features for one station. I picked Torvegade because it was a mid-sized station. I then analyse its errors and motivate the use of engineered features and ridge regression. Next, I investigate whether adding a lag feature improves prediction further. Finally, I evaluate the best model on the test data, compare pooled and station-specific models across all stations, and evaluate a Torvegade-initialized model on the Dronning Louises Bro dataset.

4.1 Baseline Model

We fit the linear regression. The features are: integer-encoded hour, day of week, month, and raw temperature (°C), wind speed (m/s) and precipitation (mm). Features are unscaled. Entries with missing bike counts are removed.

4.1.1 Analysis of the errors

The mean RMSE is 271. The model is unable to handle spikes and valleys 1. This may stem partly from not representing cycles well, which motivates cyclic features. Looking at the plot of residuals against temperature 2, split by weekday/weekend indicator variables, we can see that interaction terms might help. In addition, the residuals exhibit fanning and a U shape, suggesting that non-linear features could help. Finally, the baseline sometimes predicts negative values, which motivates clipping the predictions.

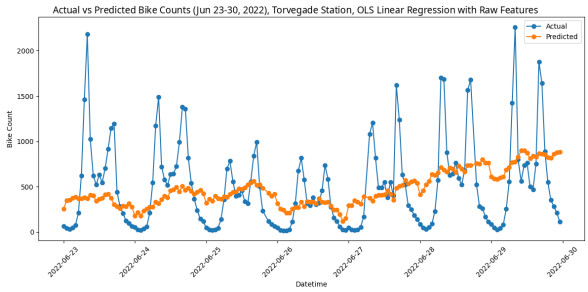


Figure 1: Predictions versus actual bicycle counts from 2022-06-23 to 2022-06-30 for the baseline linear regression model.

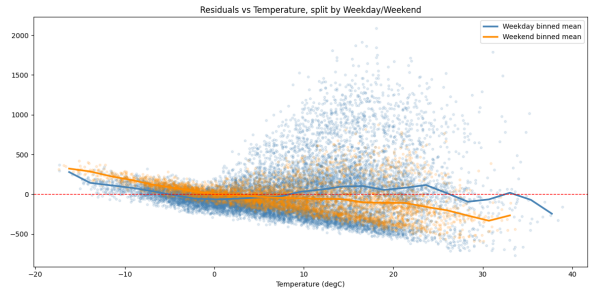


Figure 2: Residual structure by temperature and weekend status, binned (25 bins) mean, for the linear regression on 2021 and 2022.

The residual analysis suggests that other features might also exhibit non-linear effects and interact with other variables. Instead of selecting such terms manually, I generate polynomial features for all variables and let the regularised model determine their importance. Ridge regression is suitable for this task because it reduces issues originating from multicollinearity.

4.2 Improved Model

The ridge regression model, trained on 77 engineered features with non-negative prediction clipping, obtains the lowest validation mean RMSE of 188 for $\lambda = 7.93$. It improves by 83 points over the baseline.

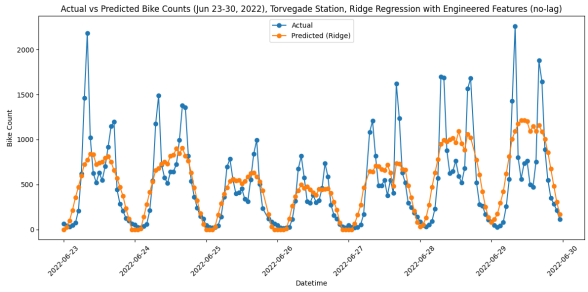


Figure 3: Predictions versus actual bicycle counts from 2022-06-23 to 2022-06-30 for the non-negative ridge regression model trained on 77 engineered features

Feature	Coefficient
temperature_C	179
hour_cos	-122
temperature_C hour_cos	-92
temperature_C^2	-64
month_cos	-62

Figure 4: Top five coefficients in ridge model, $\lambda = 7.93$, non-negative predictions, 77 features (without lag features). Coefficients are rounded to the nearest whole number.

New features contribute substantially to the predictions 4. However, the model is still unable to fully account for weekly patterns 3.

4.2.1 Temporal Model (5.B)

Lag features are added per 3.4 (90 features). The model obtains a validation mean RMSE of 131, improvement of 57 points over the (non-lag) improved model. This configuration will be called 'the best model', and is used for testing 4.3. The model predictions for most of the time follow the actual counts 5.

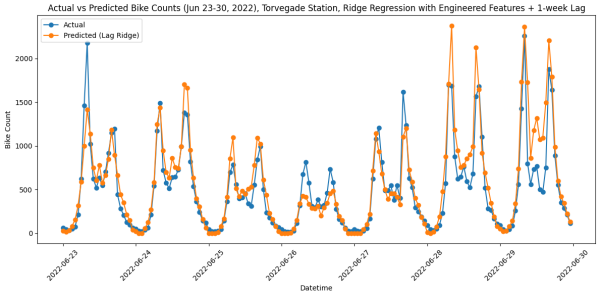


Figure 5: Predictions versus actual bicycle counts from 2022-06-23 to 2022-06-30 for the ridge regression model with 90 engineered features, including 168h lag features.

4.3 Generalisation

Relatively small degradation is observed between the pooled model (trained on all stations) and station-specific models on each station 1. In both setups, the station-wise RMSE varies substantially.

A model initialized on the Torvegade station and evaluated on the Dronning Louises Bro test dataset obtains test mean RMSE of 245. Since the walk-forward evaluation appends target-station weeks after prediction, this should be interpreted as transfer with weekly adaptation rather than as a fixed cross-station model.

The Torvegade validation mean RMSE is also close to the Torvegade test RMSEs.

Station (test)	Pooled RMSE	Specific RMSE
Dronning Louises Bro	243.27	226.02
Torvegade	132.00	127.88
Noerrebrogade	189.59	182.38
H.C. Andersens Boulevard	111.54	104.63
Langebro	100.93	90.45
Amagerbrogade	93.21	77.44

Table 1: Comparison of pooled and station-specific mean RMSE on each station's test dataset. Ridge regression with $\lambda = 163$, non-negative predictions and 90 features.

5 Discussion

The best model captures the main trends and does not appear to overfit, with a Torvegade RMSE on the order of 130–150 cyclists per hour. This error scale may be acceptable for aggregate planning or CO_2 estimation, but peak-hour traffic management would require more careful assessment.

Feature engineering clearly helped: among the top five coefficients, there is both an interaction term and a squared term. The coefficients are also plausible, although they should be interpreted cautiously because polynomial features are correlated and standardised. These results suggest that bicycle counts are driven by non-linear and cyclical relationships, and that the feature-expanded linear approach is therefore justified.

The pooled model performs slightly worse than the station-specific models. This suggests that cyclists passing through different stations respond similarly to weather and time, which is unsurprising. At the same time, some station-specific information is missing from the available features. This may be due to differences in baseline traffic levels across stations, and a pooled model with station indicators could therefore be a useful extension. Given the low training cost and the better performance, station-specific models should be preferred for the current feature set.

However, the approach has several limitations, most of them originating from the data. The analysis uses observed weather, whereas a deployed model would have to rely on weekly forecasts. In addition, the data were collected during the COVID-19 pandemic, which limits transferability beyond 2021–2023. The data also do not capture influential events such as concerts, demonstrations, festivals, or marathons, which could completely distort the counts. Finally, because the model uses lag features, it requires continuous weekly data, which may not always be available; the interpolation strategy used here should therefore be rechecked under a stricter real-time setting.

Future work could include an ablation study, for example comparing weather/calendar-only, lag-only, and multi-lag models, or checking whether ridge regression was truly necessary. Other useful extensions would be station indicators in the pooled model, normalized station-wise error metrics, deeper inspection of the regression assumptions, and comparison with other regressors.

6 Conclusions

I have shown that a ridge regression model based on weather, calendar, and lag features is a promising baseline for predicting bike counts in the near future. However, it should not be consid-

ered deployment-ready before being re-analysed with forecast weather, newer data, and a stricter real-time treatment of lag features and missing values.

References

[1] Copenhagen Municipality, *The Bicycle Account 2022 — Copenhagen City of Cyclists*, 2022. https://kk.sites.itera.dk/apps/kk_pub2/pdf/2420_d4db2492337f.pdf

[2] Copenhagen Municipality Open Data, *Fixed Bicycle Counts (Faste Cykeltællinger)*. https://www.opendata.dk/city-of-copenhagen/faste-cykeltaellinger

[3] Danish Meteorological Institute, *Open Data API*. https://opendatadocs.dmi.govcloud.dk/

[4] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*, Springer, 2023. https://link.springer.com/book/10.1007/978-3-031-38747-0

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[6] scikit-learn developers, *TimeSeriesSplit — scikit-learn stable documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

A Appendix

A.1 Figures

Station	Non-missing counts	Missing (%)
Amagerbrogade	25,779	1.91
Dronning Louises Bro	25,794	1.85
H.C. Andersens Boulevard	25,720	2.13
Langebro	25,766	1.96
Noerrebrogade	25,760	1.98
Torvegade	25,728	2.10

Table 2: Availability of bicycle counts by station. Each station has 26,280 total hourly observations. Missing percentages are rounded to two decimal places.

Split	Fold structure	Fold duration
Training	103 testing folds; first fold used only for training	164 hours per testing fold ($6.8\overline{3}$ days); first training-only fold: 237 hours (9.875 days)
Test	Weekly rolling evaluation folds	168 hours per fold (7 days)

Table 3: Fold sizes used in the time-series evaluation.

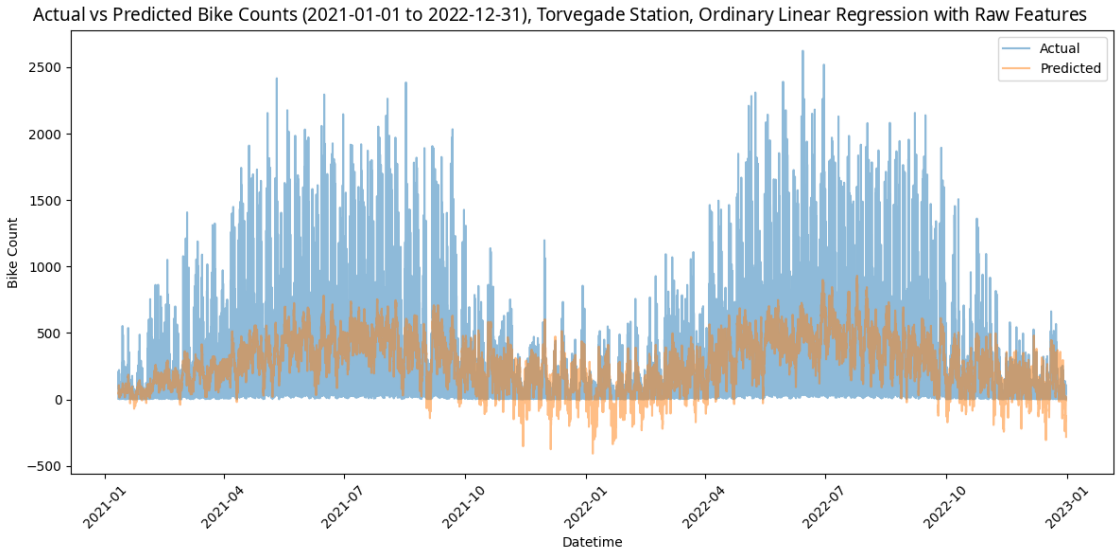


Figure 6: Predictions versus actual bicycle counts from 2021-01-01 to 2022-12-31 for the baseline linear regression model.

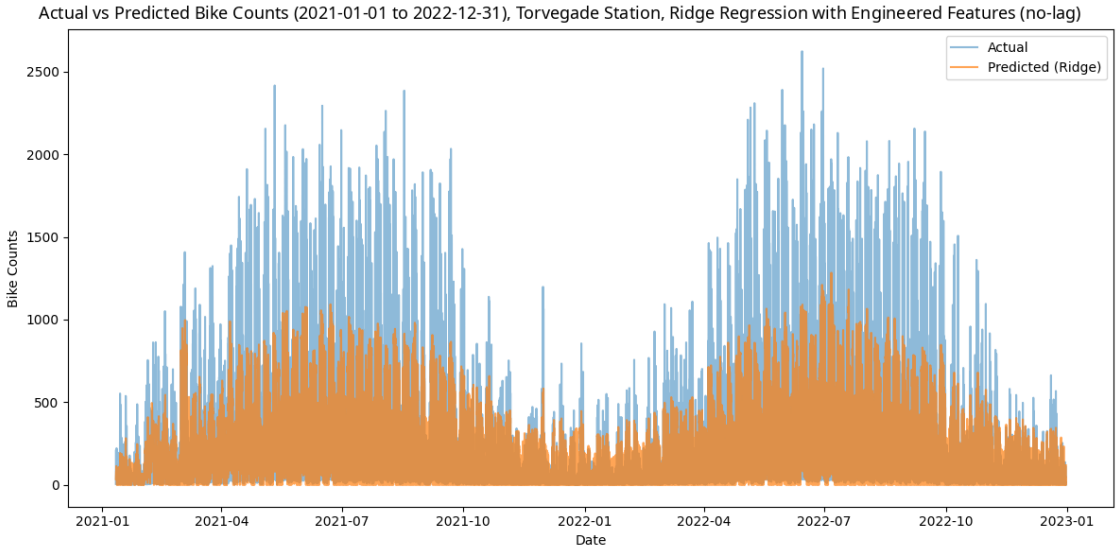


Figure 7: Predictions versus actual bicycle counts from 2021-01-01 to 2022-12-31 for the ridge regression model without lag features.

A.2 Use of Generative AI

I used Claude Code and Github Copilot mostly to create visualisations and help me with debugging. All ideas/decisions were mine. (The single exception was the idea to plot residual values against independent variable, instead of fitted values only to motivate the polynomial features). Outline of the report and the report have been written by me. OpenAI's Prism was used for proofreading, formatting, and figure creation. Claude summarised my pipeline.

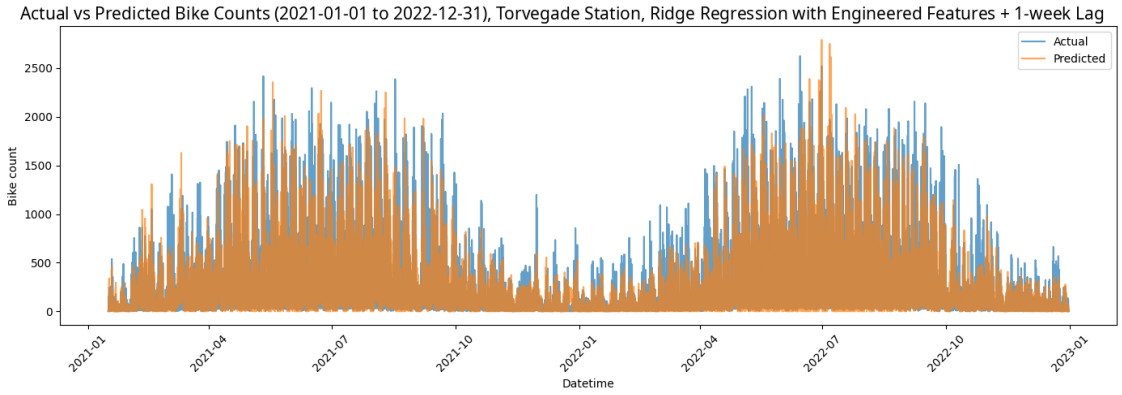


Figure 8: Predictions versus actual bicycle counts from 2021-01-01 to 2022-12-31 for the ridge regression model with the one-week lag feature.

A.3 Pipeline details

Final pipeline

The best model is built as a single sklearn `Pipeline`, mapping a raw feature row $\mathbf{r}$ to a non-negative prediction $\hat{y}$:

$$\mathbf{r} \xrightarrow{\text{encode}} \mathbf{x} \xrightarrow{\text{poly}} \boldsymbol{\phi}(\mathbf{x}) \xrightarrow{\text{scale}} \mathbf{z} \xrightarrow{\text{Ridge}} \tilde{y} \xrightarrow{\text{clip}} \hat{y}$$

```
def make_bike_pipeline_lag(alpha=163):
    feature_transformer = ColumnTransformer([
        ("cyclic",  DatetimeCyclicEncoder(), [DATETIME_COL]),
        ("numeric", "passthrough", WEATHER_COLS + [LAG_COL]),
        ("binary",  "passthrough", BINARY_COLS),
    ], remainder="drop")
    return Pipeline([
        ("features", feature_transformer),
        ("poly",     PolynomialFeatures(degree=2, include_bias=False)),
        ("scaler",   StandardScaler()),
        ("model",    ClippedRidge(alpha=alpha)),
    ])
```

(1) Feature encoding. The raw row contains a timestamp, three weather variables, a one–week lag of the bike count, and a holiday flag,

$$\mathbf{r} = \big(t,\ T,\ w,\ p,\ \ell_t,\ h\big), \qquad \ell_t = y_{t-168\text{h}}.$$

Hour, day-of-week and month are encoded cyclically; `is_weekend` is derived internally:

```
class DatetimeCyclicEncoder(BaseEstimator, TransformerMixin):
    def transform(self, X):
        dt   = pd.Series(X.iloc[:, 0])
        hour, dow, mo = dt.dt.hour, dt.dt.dayofweek, dt.dt.month
        return np.column_stack([
            np.sin(2*np.pi*hour/24), np.cos(2*np.pi*hour/24),
            np.sin(2*np.pi*dow/7),   np.cos(2*np.pi*dow/7),
            np.sin(2*np.pi*mo/12),   np.cos(2*np.pi*mo/12),
            (dow >= 5).astype(float),
        ])
```

This produces a 12-dimensional encoded vector

$$\mathbf{x} = \underbrace{(s_h, c_h, s_d, c_d, s_m, c_m, \mathbb{1}_{\text{wknd}})}_{\text{7 calendar}} \,\|\, \underbrace{(T, w, p)}_{\text{weather}} \,\|\, \underbrace{\ell_t}_{\text{lag}} \,\|\, \underbrace{h}_{\text{holiday}} \;\in\; \mathbb{R}^{12}.$$

(2) Polynomial expansion. `PolynomialFeatures(degree=2, include_bias=False)` adds all squares and pairwise products,

$$\boldsymbol{\phi}(\mathbf{x}) = (x_i)_i \,\|\, (x_i x_j)_{i \le j} \in \mathbb{R}^d, \qquad d = 12 + \binom{13}{2} = 90.$$

(3) Standardisation. Each polynomial feature is centred and scaled using statistics from the training fold only: $z_k = (\phi_k - \mu_k)/\sigma_k$.

(4) Ridge regression with non-negativity clipping. The custom estimator wraps `Ridge` and clips the output:

```
class ClippedRidge(BaseEstimator, RegressorMixin):
    def fit(self, X, y):
        self.ridge_ = Ridge(alpha=self.alpha).fit(X, y); return self
    def predict(self, X):
        return np.clip(self.ridge_.predict(X), 0, None)
```

The fitted coefficients minimise the regularised squared loss with $\lambda = 163$:

$$(\hat{\boldsymbol{\beta}}, \hat{\beta}_0) = \arg\min_{\boldsymbol{\beta}, \beta_0} \sum_{n=1}^{N} \big(y_n - \beta_0 - \mathbf{z}_n^\top \boldsymbol{\beta}\big)^2 + \lambda \|\boldsymbol{\beta}\|_2^2,$$

and the final prediction is

$$\hat{y} = \max\big(0,\ \hat{\beta}_0 + \mathbf{z}^\top \hat{\boldsymbol{\beta}}\big).$$

Training protocol. λ is chosen by `TimeSeriesSplit` CV on 2021–2022. Test performance (2023) is reported via *walk-forward* evaluation: the pipeline is refit each test week with the preceding test weeks appended to the training set.